INDEPENDENT SECURITY RESEARCH
WEB APPLICATION PENETRATION TEST

REPORT REFERENCE	[SAMPLE] PENTEST-2026-XXX
CLASSIFICATION	CONFIDENTIAL — ANONYMISED SAMPLE

SECURITY ASSESSMENT REPORT

TargetApp Inc.

Web Application Penetration Test

Full black-box and white-box penetration test including infrastructure reconnaissance, port analysis, and deep source code review of the TargetApp Inc. e-commerce platform.

TARGET APPLICATION	`app.targetapp.local`
TEST DATE	[Month Year]
FINDINGS	5 Vulnerabilities (3 Critical / High)
RESEARCHER	Kaif Shaikh (alias: Magician Slime)

CONFIDENTIAL — CLIENT USE ONLY

SECTION 01

Executive Summary

A comprehensive penetration test was conducted against the TargetApp Inc. web application at app.targetapp.local. The assessment combined infrastructure-level reconnaissance with deep source code analysis of the provided codebase. Five security vulnerabilities were identified — two of which are critically exploitable with no authentication required beyond a standard user account.

TOTAL FINDINGS	CRITICAL / HIGH
5	**3**
Across all severity levels	Require immediate remediation

INFRASTRUCTURE RISK	CODE-LEVEL RISK
Low	**Critical**
HTTP/HTTPS only, no exposed services	Business logic fully bypassable

OVERALL RISK SCORE (CVSS AGGREGATED) — 7.6 — HIGH: IMMEDIATE ACTION REQUIRED

- The combination of client-side price manipulation (CVSS 9.3), an insecure split-payment RPC, and client-controlled coupon usage creates a complete payment bypass chain.
- Any registered user can purchase any product for effectively $0 (or local currency equivalent).

SECTION 02

Test Methodology

The assessment followed a structured multi-phase approach, combining automated reconnaissance with manual expert analysis of both the live application and its source code.

1. PHASE 1 — RECONNAISSANCE: DNS resolution performed on app.targetapp.local to identify the underlying IP infrastructure. Target was hosted on a CDN edge network. IP geolocation and ASN lookup confirmed CDN hosting — no bare-metal exposure.
2. PHASE 2 — PORT & SERVICE SCANNING: TCP port scan against the resolved IP. Only ports 80 (HTTP) and 443 (HTTPS) were found open. No unexpected services (SSH, RDP, database ports, admin panels) were exposed. Infrastructure attack surface is minimal.
3. PHASE 3 — SOURCE CODE ACQUISITION: Full application source code retrieved, enabling white-box analysis. This supplemented dynamic testing with static analysis of API routes, database functions, and client-side logic.

4. PHASE 4 — STATIC CODE ANALYSIS: Manual expert review of TypeScript API routes, React frontend components, PostgreSQL database migration functions, and React hooks. Focus placed on payment flows, coupon validation, and webhook handling.
5. PHASE 5 — REPORTING: All identified vulnerabilities documented with source code evidence, attack scenario walkthroughs, estimated business impact, and prioritised remediation steps.

SECTION 03

Scope of Assessment

The following targets and components were included in and excluded from this assessment.

ASSET / COMPONENT	TYPE	STATUS	NOTES
https://app.targetapp.local	Web Application	IN SCOPE	Primary target, full application
api/payment-createorder.ts	Serverless API Route	IN SCOPE	Payment order creation endpoint
api/payment-webhook.ts	Serverless API Route	IN SCOPE	Payment gateway callback handler
api/payment-verify.ts	Serverless API Route	IN SCOPE	Payment signature verification
src/hooks/useCoupon.ts	Frontend Hook	IN SCOPE	Coupon validation & usage tracking
database/migrations/	DB Functions (PostgreSQL)	IN SCOPE	Split-payment RPC, RLS policies
Payment Gateway Admin Dashboard	Third-party SaaS	OUT OF SCOPE	Not authorised
Payment Gateway Infrastructure	Third-party Service	OUT OF SCOPE	Not authorised
Social Engineering	Attack Vector	OUT OF SCOPE	Not in agreement

Vulnerability Findings

All findings were identified through manual source code review. Severity ratings follow the CVSS v3.1 scoring standard. Findings are ordered by severity.

TAPP-2026-001

Business Logic Flaw: Client-Side Price Manipulation in Payment Flow

[CRITICAL] CVSS 9.3 OWASP A04 — Insecure Design

DESCRIPTION

The application's payment initiation flow contains a fundamental architectural flaw: the purchase amount is calculated entirely on the client (browser) and sent to the backend API as a trusted value. The backend endpoint POST /api/payment-create-order accepts the amount field from the HTTP request body and uses it verbatim to create a payment gateway order — with no server-side lookup of the actual product price from the database.

An attacker intercepts the outgoing POST request using a proxy tool (e.g. Burp Suite, browser DevTools), modifies the amount field to any value (e.g. $0.01), and the backend creates a legitimate payment order for that manipulated amount. Upon completing the minimal payment, the product order is permanently marked as completed and access is granted.

VULNERABLE CODE

API/PAYMENT-CREATE-ORDER.TS

```
// Request body arrives from the browser — fully attacker-controlled
const { productId, productTitle, amount } = req.body;

// ✗ No database lookup. No price validation. Amount is blindly trusted.
const amountInCents = Math.round(amount * 100);

// A payment order is now created for whatever amount the attacker sent.
const orderRes = await fetch('https://api.payment-gateway.com/v1/orders', {
  body: JSON.stringify({ amount: amountInCents, currency: 'USD' })
});
```

ATTACK SCENARIO (STEP-BY-STEP)

EXPLOIT CHAIN — PRODUCT PURCHASED FOR $0.01

1. Login as any registered user
2. Click "Buy Now" on a $49.99 product
3. Intercept the outgoing POST /api/payment-create-order request
4. Modify request body: amount: 49.99 → 0.01
5. Complete payment of $0.01 via the payment gateway
6. Order completion triggered ✓ — Product access granted at manipulated price

BUSINESS IMPACT

DIRECT REVENUE LOSS

- Every product on the platform can be purchased for $0.01 (or any arbitrary amount) by any registered user.

- Vendors / content authors receive incorrect or zero earnings based on manipulated purchase records.
- This attack requires no technical sophistication — any browser extension that intercepts requests is sufficient.
- The exploit is completely silent — no server error is thrown, no alert is triggered. The attacker receives a legitimate purchase record.

REMEDIATION

HOW TO FIX

✓ Never accept price from the client. The API should only accept productId and, optionally, couponCode.

✓ In payment-create-order.ts, query the database server-side: SELECT price_amount FROM product_catalog WHERE id = productId.

✓ Apply coupon discount server-side after validating the coupon against the database. Calculate the final amount in the backend, never the frontend.

✓ Store the authoritative price in the product_purchases record and validate against it on webhook receipt.

TAPP-2026-002

Insecure Split-Payment RPC: No Server-Side Price Verification in PostgreSQL Function

[HIGH] CVSS 8.1 OWASP A04 — Insecure Design

DESCRIPTION

The platform supports split payments where a portion is paid using the in-app currency "StoreCredits" and the remainder via cash. This flow is handled by the PostgreSQL stored procedure purchase_product_split_payment, which is callable directly from the JavaScript SDK using client.rpc().

The critical flaw is that the function accepts p_credits_amount and p_cash_amount as fully caller-controlled parameters without ever querying the actual product price from product_catalog to verify that the sum matches the real price. An attacker can call this RPC directly with both parameters set to near-zero values and receive full product access.

VULNERABLE CODE

DATABASE/MIGRATIONS/SPLIT_PAYMENT_FUNCTION.SQL

```
CREATE OR REPLACE FUNCTION public.purchase_product_split_payment(
  p_product_id   UUID,
  p_product_title TEXT,
  p_credits_amount NUMERIC,  -- ✕ Fully attacker-controlled
  p_cash_amount    NUMERIC   -- ✕ Fully attacker-controlled
) ...
DECLARE
  v_total NUMERIC := p_credits_amount + p_cash_amount;  -- ✕ No price DB lookup
BEGIN
  -- Function never queries:
  --   SELECT price FROM product_catalog WHERE id = p_product_id
  -- Immediately proceeds to debit StoreCredits and mark purchase as completed.
  INSERT INTO public.product_purchases (...) VALUES (v_user_id, p_product_id, v_total,
'completed', ...);
```

ATTACK SCENARIO (STEP-BY-STEP)

EXPLOIT CHAIN — FREE PRODUCT VIA DIRECT RPC CALL

1. Obtain the client SDK anon key (publicly visible in the JavaScript bundle)
2. Authenticate as a standard registered user
3. Call client.rpc('purchase_product_split_payment', { p_credits_amount: 0, p_cash_amount: 0.01, ... })
4. Purchase inserted as 'completed' for $0.01
5. Full product access granted ✓ — payment gateway never contacted

BUSINESS IMPACT

REVENUE BYPASS VIA DIRECT DATABASE CALL

- Attacker bypasses the payment gateway entirely — no real payment is ever processed.
- Products can be obtained for as little as $0.01 without the frontend ever being involved.

- The client SDK anon key is intentionally public (in the JavaScript bundle) — only RLS policies and function security prevent abuse. This RPC does not verify amounts, making RLS insufficient here.
- Can be scripted to bulk-acquire the entire product library for fractions of a cent.

REMEDIATION

HOW TO FIX

✓ At the start of the SQL function, query the real price: SELECT price_amount INTO v_product_price FROM product_catalog WHERE id = p_product_id.

✓ Add an integrity check before proceeding: IF (p_credits_amount + p_cash_amount) < v_product_price THEN RAISE EXCEPTION 'Payment amount insufficient'; END IF;

✓ Also verify the user's actual StoreCredits balance server-side before debiting — confirm they genuinely hold the credits claimed in p_credits_amount.

TAPP-2026-003

Insecure Direct Object Reference (IDOR): Coupon Usage Counter Controlled by Client

[HIGH] CVSS 7.5 OWASP A01 — Broken Access Control

DESCRIPTION

Coupon codes support a max_uses limit to prevent reuse. The platform enforces this limit during coupon validation (checking uses_count < max_uses), but the actual incrementing of the usage counter is performed as a separate, optional client-side call to the database after the payment completes.

Because this increment is not part of an atomic server-side transaction, an attacker who completes payment simply never calls incrementUsage(), leaving the coupon counter unchanged and reusable indefinitely. Additionally, the implementation has a classic read-modify-write race condition: if two requests apply the same coupon simultaneously, both read the same uses_count and both write back count + 1, resulting in only one increment for two coupon uses.

VULNERABLE CODE

SRC/HOOKS/USECOUPON.TS

```
// ✗ Two separate DB calls — exploitable read-modify-write race condition
const incrementUsage = async (couponId: string) => {
  const { data } = await db
    .from("coupon_codes")
    .select("uses_count")
    .eq("id", couponId)
    .single();

  const current = (data as any)?.uses_count ?? 0;

  await db
    .from("coupon_codes")
    .update({ uses_count: current + 1 })
    .eq("id", couponId);
};
```

SRC/PAGES/PRODUCTDETAIL.TSX — INCREMENT IS OPTIONAL, CALLED AFTER PAYMENT

```
await initiatePayment.mutateAsync({ productId, productTitle, amount: finalAmount });

// ✗ This line can be blocked by an attacker using a proxy.
// If it is never sent, the coupon limit is never enforced.
if (appliedCoupon) await incrementUsage(appliedCoupon.id);
```

BUSINESS IMPACT

COUPON ABUSE — UNLIMITED DISCOUNT EXPLOITATION

- A one-time or limited-use coupon (e.g. 50% off for 10 uses) can be used an unlimited number of times by any user who intercepts and drops the increment call.
- Discount codes meant for marketing campaigns or single-user promotions become effectively infinite.
- The race condition means even honest concurrent checkouts can silently under-count usage by multiple uses at once.

- Combined with Finding #1: attacker applies a 100% discount coupon, sends amount=0.01, and completes an order for $0.01 — coupon remains available for further abuse.

REMEDIATION

HOW TO FIX

✓ Move coupon increment to the backend as an atomic SQL operation: UPDATE coupon_codes SET uses_count = uses_count + 1 WHERE id = $1 AND uses_count < max_uses RETURNING id. Use the returned row count to confirm the coupon was still valid at time of use.

✓ Perform this increment inside the payment verification function or a database trigger, not from the frontend. It must be atomic with the purchase record insertion.

✓ Consider a database-level CHECK constraint: CHECK (uses_count <= max_uses) as a last-resort safety net.

TAPP-2026-004

Conditional Webhook Signature Verification — Missing Secret Bypasses Auth

[MEDIUM] CVSS 6.5 OWASP A07 — Identification & Auth Failures

DESCRIPTION

The payment webhook handler at api/payment-webhook.ts wraps its HMAC-SHA256 signature verification inside a conditional block that only executes if the environment variable PAYMENT_WEBHOOK_SECRET is present. If the variable is absent — due to a misconfiguration, a secret rotation mistake, or a deployment error — the endpoint silently skips all verification and processes any incoming webhook payload as legitimate.

This means a simple HTTP POST with a crafted payment.captured event and a valid order_id would mark any pending purchase as completed — without any money ever being paid — if the secret is ever missing.

VULNERABLE CODE

API/PAYMENT-WEBHOOK.TS

```
const PAYMENT_WEBHOOK_SECRET = process.env.PAYMENT_WEBHOOK_SECRET;

// ✗ Verification is conditional — silently skipped if env var is absent
if (PAYMENT_WEBHOOK_SECRET) {
  const expectedSig = crypto
    .createHmac('sha256', PAYMENT_WEBHOOK_SECRET)
    .update(rawBody)
    .digest('hex');
  if (expectedSig !== signature) {
    return res.status(400).json({ error: 'Invalid signature' });
  }
} else {
  console.warn('Skipping signature verification');  // ✗ Should be HTTP 500
  // Execution continues — webhook payload is processed as trusted!
}
```

BUSINESS IMPACT

FAKE PAYMENT INJECTION (CONDITIONAL EXPLOITABILITY)

- If PAYMENT_WEBHOOK_SECRET is ever missing or empty, any attacker can send a forged payment.captured event and grant product access without payment.
- Environment variable misconfiguration is common during deployments, secret rotations, or team onboarding.
- Risk is Low today (secret is presumably configured) but becomes Critical the moment it disappears — with no alerting mechanism in place.

REMEDIATION

HOW TO FIX

✓ Make signature verification unconditional. If PAYMENT_WEBHOOK_SECRET is missing at startup, return HTTP 500 immediately — do not proceed.

✓ Replace the conditional block with: if (!PAYMENT_WEBHOOK_SECRET) return res.status(500).json({ error: 'Webhook secret not configured' });

✓ Add an alert or monitoring rule to fire if the webhook endpoint returns 5xx — this would catch a missing secret immediately.

TAPP-2026-005

Permissive CORS Policy — Wildcard Origin on API Routes

[LOW] **CVSS 4.3** OWASP A05 — Security Misconfiguration

DESCRIPTION

Three of the four API routes (payment-create-order.ts, payment-verify.ts, extract-data.ts) set Access-Control-Allow-Origin: *, permitting any website on the internet to make cross-origin requests to these endpoints. While these endpoints require an authenticated JWT, the wildcard origin expands the attack surface for CSRF-adjacent attacks and makes it easier for a malicious third-party site to exploit authenticated users.

VULNERABLE CODE

API/PAYMENT-CREATE-ORDER.TS (SAME PATTERN IN PAYMENT-VERIFY.TS, EXTRACT-DATA.TS)

```
res.setHeader('Access-Control-Allow-Origin', '*');          // ✕ Any origin permitted
res.setHeader('Access-Control-Allow-Credentials', 'true');
res.setHeader('Access-Control-Allow-Methods', 'GET,OPTIONS,POST');
```

BUSINESS IMPACT

EXPANDED ATTACK SURFACE

- Any website can make authenticated requests to these API routes if the user's JWT is accessible in a cross-origin context.
- Combined with XSS vulnerabilities (if introduced in the future), a wildcard CORS policy allows complete session hijacking from third-party origins.
- Severity is currently Low due to JWT authentication, but will elevate if additional vulnerabilities are introduced.

REMEDIATION

HOW TO FIX

✓ Replace * with the explicit production domain: Access-Control-Allow-Origin: https://app.targetapp.local

✓ For local development, read an allowlist from an environment variable and validate the request's Origin header against it before reflecting.

✓ Remove Access-Control-Allow-Credentials: true from any route that also uses a wildcard — this combination is rejected by browsers anyway and signals misconfigured intent.

Risk Summary Matrix

Consolidated view of all findings, their CVSS scores, and recommended remediation priority.

ID	VULNERABILITY	SEVERITY	CVSS	OWASP CATEGORY	PRIORITY
TAPP-001	Client-Side Price Manipulation	CRITICAL	9.3	A04 Insecure Design	Immediate
TAPP-002	Split-Payment RPC No Price Verify	HIGH	8.1	A04 Insecure Design	Immediate
TAPP-003	IDOR: Coupon Usage Client-Controlled	HIGH	7.5	A01 Broken Access Control	Immediate
TAPP-004	Conditional Webhook Sig Verification	MEDIUM	6.5	A07 Auth Failures	Short-term
TAPP-005	Permissive CORS Wildcard Origin	LOW	4.3	A05 Security Misconfiguration	Medium-term

Strategic Recommendations

Beyond individual bug fixes, the following architectural changes will significantly improve the platform's long-term security posture.

01

Server is the Source of Truth for Pricing

The frontend should only ever send identifiers (productId, couponCode). All pricing, discount calculation, and final amount determination must happen on a trusted server with database access. Treat every client-provided numeric value as hostile input.

• IMMEDIATE PRIORITY

02

Atomic Transactions for All Purchase Logic

Coupon increment, StoreCredits debit, and purchase record creation should all happen inside a single database transaction. If any step fails, all steps roll back. This eliminates race conditions and ensures data consistency.

• IMMEDIATE PRIORITY

03

Mandatory Webhook Signature Verification

Payment webhooks must always verify HMAC signatures. A missing secret should hard-fail the deployment, not silently disable security. Use a secret management service and alert on any 5xx response from the webhook endpoint.

• SHORT-TERM PRIORITY

04

Restrict CORS to Known Origins

Lock all API routes to the production domain and a developer allowlist. Avoid wildcard origins on any endpoint that processes payments or user data, even if those endpoints require authentication.

• MEDIUM-TERM PRIORITY

05

Add Purchase Amount Integrity Check

Add a database-level CHECK constraint or trigger that prevents a product_purchases record from being marked 'completed' if the stored amount does not match the product's price_amount at time of purchase. This is a last-resort safety net.

• SHORT-TERM PRIORITY

06

Restrict RPC Access via Row-Level Security

Ensure the purchase_product_split_payment function cannot be called directly from the client without passing server-side validation. Route all payment RPCs through a backend edge function that adds an extra verification layer.

- **SHORT-TERM PRIORITY**

K

Kaif Shaikh

alias: Magician Slime

Independent Security Researcher & Web Penetration Tester

magicianslime.netlify.app

Assessment Type	Report Date	Report Reference
Black-box Recon + White-box Source Code Review	**[Month Year]**	**[SAMPLE] PENTEST-2026-XXX**

This sample report reflects the format, structure, and technical depth used in a real signed security assessment engagement. All client-specific details have been anonymised. Findings represent real vulnerability classes discovered during an actual assessment. No production systems of the anonymised client were harmed during testing.

[SAMPLE] PENTEST-2026-XXX | Prepared by Kaif Shaikh (Magician Slime)